Transformer Implementation from Scratch for Machine Translation and Finetuning Pre-trained for VLSP 2025 Medical Domain

Phạm Tuấn Anh 23020010@vnu.edu.vn
Phan Bá Thọ 23020136@vnu.edu.vn
Đoàn Thái Hùng 23021565@vnu.edu.vn
Dương Khôi Nguyên 23020127@vnu.edu.vn

Tóm tắt nội dung

This report presents our group's work for the NLP 2025 course final project. The project consists of two main parts. First, we implement a Sequence-to-Sequence Machine Translation model based on the Transformer architecture entirely from scratch (70% of the project). We detail the construction of core components such as Multi-Head Attention, Positional Encoding, and Feed-Forward Networks, along with the full training pipeline. Second, we participate in the VLSP 2025 Shared Task on Machine Translation in the Medical Domain (30%). For this task, we fine-tune a pre-trained Large Language Model (Qwen) specifically allowed by the organizers, adapting it to the medical domain constraints. We provide a comprehensive report on data preprocessing, model statistics, architectural details, and evaluation results using BLEU and Gemini Score.

1 Introduction

Machine Translation (MT) has seen significant advancements with the introduction of the Transformer architecture. In this project, we aim to deeply understand this architecture by implementing it from scratch. Furthermore, we address the challenges of domain adaptation by applying a pre-trained model to the medical domain as part of the VLSP 2025 Shared Task. This report is organized into two main parts: Part I details our implementation of the Transformer from scratch, and Part II describes our adaptation of the Qwen model for the medical domain.

2 Preliminaries

Before detailing our implementation, we briefly define the Neural Machine Translation (NMT) problem and the notation used throughout this report.

2.1 Problem Formulation

Given a source sequence $X = (x_1, x_2, \ldots, x_n)$ and a target sequence $Y = (y_1, y_2, \ldots, y_m)$, the goal of an NMT model is to learn the conditional probability distribution $P(Y|X)$. By the chain rule of probability, this can be decomposed as:

$$P(Y|X) = \prod_{t=1}^{m} P(y_t|y_{<t}, X)$$

where $y_{<t}$ denotes the history of generated tokens $(y_1, \ldots, y_{t-1})$.

2.2 Transformer Overview

The Transformer (Vaswani et al., 2017) is a sequence-to-sequence architecture that relies entirely on attention mechanisms, dispensing with recurrence and convolutions. It follows an Encoder-Decoder structure:

- **Encoder:** Maps the input sequence X into a sequence of continuous representations $Z = (z_1, \ldots, z_n)$.
- **Decoder:** Generates the output sequence Y one element at a time, consuming Z and the previously generated symbols.

3 Part I: Transformer from Scratch

In this section, we describe our implementation of a Sequence-to-Sequence Machine Translation model based on the Transformer architecture, built entirely from basic components without high-level libraries.

3.1 Data Preparation (IWSLT)

3.1.1 Dataset

We utilized the IWSLT 2015 English-Vietnamese dataset (Cettolo et al., 2015), sourced from

Kaggle[1]. This dataset is widely used for low-resource machine translation research. The dataset is split into:

- Training: train.en.txt and train.vi.txt (approx. 133k sentence pairs).
- Validation: tst2012.en.txt and tst2012.vi.txt (1,553 pairs).
- Test: tst2013.en.txt and tst2013.vi.txt (1,268 pairs).

Each file consists of sentences. Each sentence in the source language corresponds line-by-line to its translation in the target language.

3.1.2 Preprocessing Pipeline

We implemented a custom preprocessing pipeline in Python to ensure data quality and compatibility with our Transformer model. The pipeline consists of the following steps:

Step 1: Text Cleaning and Tokenization. We use a simple tokenizer that goes through the input sentence character by character. It keeps only letters, numbers, and spaces, and adds spaces around basic punctuation marks such as periods, commas, and question marks so they become separate tokens. Uncommon or non-standard characters (e.g., }, , %) are removed. The final token list is obtained by splitting the processed string by whitespace. This method is straightforward and does not rely on external libraries.

Step 2: Vocabulary Construction. We built separate vocabularies for English (source) and Vietnamese (target) from the training data. Word frequencies were counted using a Counter, and tokens appearing fewer than five times were removed to limit noise and reduce vocabulary size. We then added special tokens for padding, unknown words, and sequence boundaries, including [PAD], [UNK], [SOS], and [EOS]. Finally, we created token-to-index and index-to-token mappings to support efficient processing during training and inference.

Step 3: Data Loading and Batching. We implemented a custom TranslationDataset class based on PyTorch's Dataset. To keep source and target data aligned, the training set was limited to the first 130,000 sentence pairs. Each sentence was truncated or padded to a fixed length of 128 tokens. For target sequences, [SOS] and [EOS] tokens were added at the beginning and end, and [PAD] tokens were used to make all sequences the same length. During data loading, padding masks and causal masks were created so the model can ignore padding tokens and prevent the decoder from seeing future tokens. Each dataset sample returns the source and target tensors along with their corresponding padding masks.

[1]https://www.kaggle.com/datasets/tuannguyenvananh/iwslt15-englishvietnamese/data

3.1.3 Data Statistics

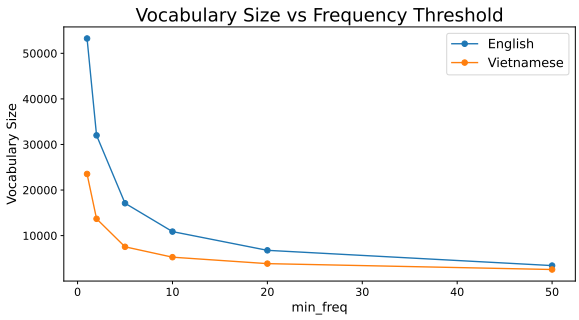

Figure 1: Vocabulary Size Comparison between English and Vietnamese after applying min_freq

Figure 1 illustrates the vocabulary sizes for English and Vietnamese after applying different minimum frequency threshold. To balance vocabulary size and coverage, we selected min_freq=10 for both languages. Table 1 summarizes the vocabulary statistics after preprocessing.

Language	Min Freq	Vocab Size
English (Source)	10	10883
Vietnamese (Target)	10	5269

Table 1: Vocabulary statistics for the IWSLT15 dataset.

Figure 2 shows that both English and Vietnamese word frequencies follow a strongly right-skewed, long-tailed distribution. Most words appear with relatively low frequencies, while only a few very common words dominate the corpus. Even with min_freq = 10, both languages retain a clear long-tail vocabulary structure.

3.2 Model Architecture

We adopt the standard Transformer architecture, which consists of an encoder stack and a decoder stack, as shown in Figure 3.

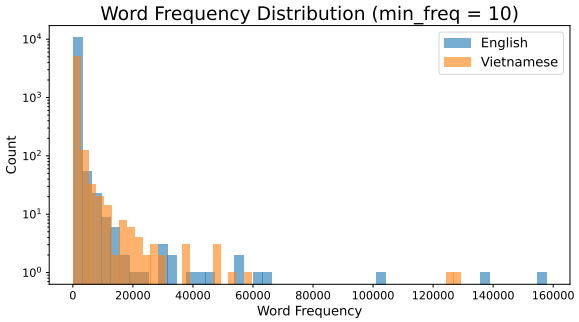

Figure 2: Word frequency distribution in English and Vietnamese vocabularies after applying min_freq=10.

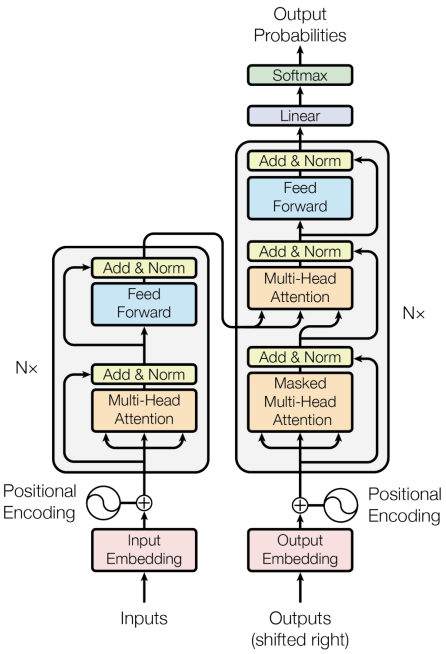

Figure 3: Transformer architecture

3.2.1 Input Embedding and Positional Encoding

The input to the model is a sequence of token indices. These indices are first passed through an Embedding Layer to convert them into dense vectors of dimension d_{model}. Since the Transformer contains no recurrence and no convolution, it is invariant to sequence order. To inject order information, we add Positional Encodings to the input embeddings. We use the sinusoidal formulation:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

This allows the model to learn to attend by relative positions.

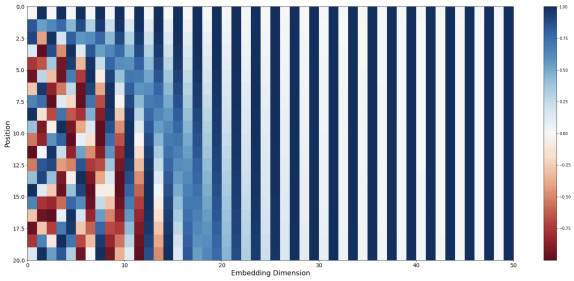

Figure 4: Visualization of the Sinusoidal Positional Encoding matrix. Each row corresponds to a position, and each column corresponds to a dimension in the embedding space.

3.2.2 Multi-Head Attention

The core component of the Transformer is the Multi-Head Attention mechanism. It allows the model to jointly attend to information from different representation subspaces at different positions.

Scaled Dot-Product Attention. The attention score is calculated as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q (Query), K (Key), and V (Value) are matrices derived from the input. The division by $\sqrt{d_k}$ prevents the dot products from growing too large in magnitude, which would push the softmax function into regions with extremely small gradients.

Multi-Head Mechanism. Instead of performing a single attention function, we project the queries, keys, and values h times with different, learned linear projections. We then perform the attention function in parallel.

Algorithm 1 summarizes the Multi-Head Attention process. Note that with self-attention, Q, K, and V all come from the same source (the input sequence) while in cross-attention, K and V come from the encoder output.

3.2.3 Position-wise Feed-Forward Network (SwiGLU)

Each layer of our encoder and decoder contains a fully connected feed-forward network. While the original Transformer used a standard two-layer ReLU network ($ReLU(xW_1 + b_1)W_2 + b_2$), we implemented the **SwiGLU** (Swish-Gated Linear Unit) variant (Shazeer, 2020).

SwiGLU is defined as:

$$SwiGLU(x) = (Swish(xW_1) \otimes (xW_3))W_2$$

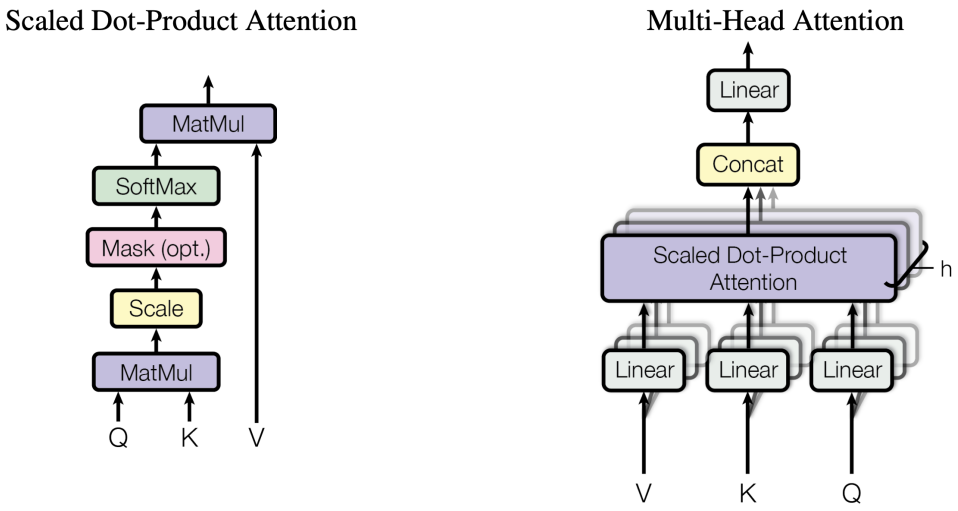

Figure 5: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Algorithm 1 Multi-Head Attention

Input: x (Input features), $mask$ (Optional)
Params: W_Q, W_K, W_V, W_O
$Q \leftarrow xW_Q$
$K \leftarrow xW_K$
$V \leftarrow xW_V$
Split Q, K, V into h heads
$Scores \leftarrow \frac{QK^T}{\sqrt{d_k}}$
if $mask$ is not None **then**
  $Scores \leftarrow Scores.fill(mask == 0, -\infty)$
end if
$Attn \leftarrow softmax(Scores)$
$Output \leftarrow Attn \times V$
Concat heads of $Output$
Return $OutputW_O$

where $\otimes$ denotes element-wise multiplication and $Swish(x) = x \cdot \sigma(x)$.

Impact of SwiGLU. SwiGLU introduces a gating mechanism that allows the model to control the flow of information more effectively. The use of the Swish activation function, which is smooth and non-monotonic, combined with the gating structure, has been shown to improve convergence and performance in Large Language Models (e.g., LLaMA, PaLM) compared to standard ReLU or GELU FFNs.

3.2.4 Encoder and Decoder Layers

Residual Connections and Layer Normalization. To facilitate gradient propagation in deep networks, we employ a residual connection (He et al., 2016) around each of the two sub-layers, followed by layer normalization (Ba et al., 2016). That is, the output of each sub-layer is:

$$LayerNorm(x + Sublayer(x))$$

Encoder Layer. The encoder is composed of a stack of N identical layers. Each layer has two sub-layers: Multi-Head Self-Attention and the SwiGLU Feed-Forward Network.

Decoder Layer and Causal. The decoder is also composed of a stack of N identical layers. In addition to the two sub-layers present in each encoder layer, the decoder inserts a third sub-layer, which performs Multi-Head Attention over the output of the encoder stack.

Crucially, the self-attention sub-layer in the decoder is modified with Causal Masking. This prevents positions from attending to future positions. We implement this by masking out (setting to $-\infty$) the upper triangular part of the attention scores matrix. This ensures that the prediction for position i depends only on the known outputs at positions less than i.

Algorithm 2 summarizes the forward pass of the Transformer model. Where the Generator is a linear layer mapping the decoder output to vocabulary logits.

3.3 Experimental Setup (Part I)

Computing environment. The experiments were conducted on a Kaggle Notebook using an NVIDIA Tesla P100 GPU with 16 GB VRAM. All models were implemented in PyTorch and trained using

Algorithm 2 Transformer Forward Pass

Input: src (Source), tgt (Target)
Params: $Encoder, Decoder, Generator$
$src\_emb \leftarrow Embedding(src) + PosEnc(src)$

$memory \leftarrow src\_emb$
for $layer$ **in** $Encoder$ **do**
  $memory \leftarrow layer(memory)$
end for
$tgt\_emb \leftarrow Embedding(tgt) + PosEnc(tgt)$

$tgt\_mask \leftarrow CreateCausalMask(tgt)$
$out \leftarrow tgt\_emb$
for $layer$ **in** $Decoder$ **do**
  $out \leftarrow layer(out, memory, tgt\_mask)$
end for

$logits \leftarrow Generator(out)$
Return $logits$

FP32 precision. This setup ensures the experiments are feasible and comparable.

Dataset and Preprocessing. The experiments use the IWSLT 2015 English–Vietnamese dataset, processing pipeline as described in 3.1. The maximum sequence length is set to 60 tokens, and sentence pairs longer than this limit are removed.

Model Architecture. We implement a Transformer encoder-decoder architecture from scratch. The model contains 111M trainable parameters. The detailed architecture configuration is presented in Tab. 2.

Component	Value
Encoder layers	6
Decoder layers	6
Embedding dimension (d_{model})	512
Feed-forward dimension (d_{ff})	2048
Attention heads (h)	8
Dropout	0.2

Table 2: Transformer architecture configuration

Training Configuration. We split data into batches of 164 and train the model using the AdamW optimizer with an initial learning rate of $3e{-}5$ and a weight decay of $1e{-}4$. Gradient clipping is applied with an ℓ_2 norm threshold of 1.0 to improve training stability. We adapt a noam scheduler (Vaswani et al., 2023) that is detailed in appendix. A.1. The learning rate is initialized at $3e{-}5$ and decayed multiplicatively at fixed step intervals, using decay rates of $[1.3, 0.95]$. A decay boundary is applied at step 3600, and the learning rate is updated every 390 steps. This design gradually reduces the learning rate throughout training without an explicit warmup phase. A visualization of the learning rate scheduler is shown in Fig. 6.

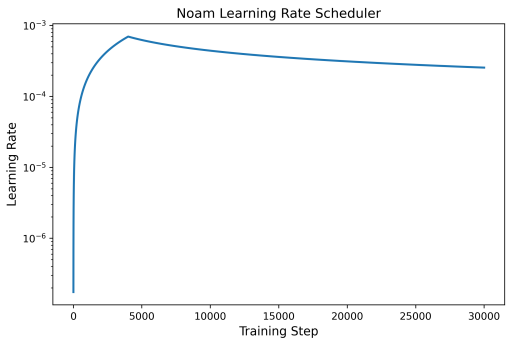

Figure 6: Step-based learning rate schedule over 30 training epochs, with an increase phase followed by decay.

Loss and Metrics. The training objective is token-level cross-entropy loss, where padding tokens are ignored during loss computation. During evaluation, translations are generated using beam search decoding instead of greedy decoding to better approximate sequence-level translation quality, using a beam size of 4 and a length penalty of 0.6, following the settings in Attention Is All You Need (Vaswani et al., 2017). For evaluation, we report token-level accuracy excluding padding tokens, perplexity (PPL), and BLEU score on the validation set using detokenized outputs. Perplexity is computed as $\mathrm{PPL} = \exp(L)$, where L denotes the average cross-entropy loss.

3.4 Results (Part I)

Training Progress. We monitored cross-entropy loss, accuracy, BLEU and perplexity on the validation set during training. Figure 7 shows the evolution of training and validation loss over epochs for the Scratch Transformer.

Evaluation Results. To avoid overfitting and ensure better generalization, we select the model checkpoint corresponding to the epoch with the lowest validation loss. Table 3 summarizes the final performance of the Scratch Transformer on the IWSLT 2015 benchmark (tst2013.en and tst2013.vi), including BLEU, accuracy, and perplexity metrics.

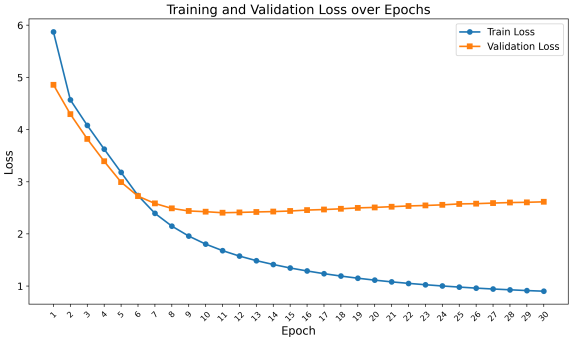

Figure 7: Training and validation loss over epochs for the Scratch Transformer.

Metric	Score
BLEU	19.12
Training Accuracy	0.625
Evaluation Accuracy	0.583
Training PPL	5.363
Validation PPL	11.089

Table 3: Performance of the Scratch Transformer on IWSLT 2015 benchmark.

The loss curves show that the model converges within the first 10 epochs. After that, the training loss keeps decreasing while the validation loss stabilizes, suggesting slight overfitting. Selecting the checkpoint with the lowest validation loss ensures better generalization. The BLEU score of 19.12, the gap between training and evaluation accuracy (0.625 and 0.583, respectively), along with training and validation perplexities of 5.36 and 11.089 show that the model generalizes reasonably well.

Limitations of the Baseline. Although the baseline Transformer provides a good starting point, it exhibits several notable limitations that affect translation quality and generalization:

- Tokenization: Word-level tokenization leads to a large vocabulary, OOV issues, limited generalization, and reliance on Vietnamese word segmentation.
- Embeddings: Embeddings are trained from scratch without pretraining, limiting semantic representation.
- Transformer: ReLU-based feed-forward network has limited expressive power and lacks a gating mechanism.

These limitations motivate a series of improvements: pretrained word embeddings using the Skip-gram model, subword tokenization via SentencePiece, and the use of SwiGLU in the feed-forward layers.

3.5 Improvements

Experimental Configuration. We conduct a series of experimental configurations to analyze the impact of different tokenization strategies and architectural enhancements on English-Vietnamese neural machine translation. Starting from a baseline, we progressively introduce incremental modifications, resulting in six configurations.

As shown in Tab. 4, we evaluate several configurations that progressively extend a word-level baseline. The baseline system employs a Keras word-level tokenizer with ViTokenizer for Vietnamese preprocessing. Config A replaces this tokenizer with SentencePiece using a shared English–Vietnamese vocabulary. Config B further adopts separate SentencePiece vocabularies for the two languages to better capture language-specific subword patterns. Building on config B, config C initializes the embedding layer with Skip-gram pretrained embeddings, which are frozen during the first two training epochs for stability. Moreover, config D replaces the ReLU-based feed-forward network in the Transformer with SwiGLU to enhance model expressiveness. In addition to these subword-based settings, we include two word-level variants with and without SwiGLU for comparison.

Training Progress. During training, we monitor the cross-entropy loss on the validation set (tst2012.en and tst2012.vi) to assess convergence behavior and optimization stability. The model checkpoint with lowest validation loss is selected for evaluation in three main metrics: BLEU, perplexity (PPL), and accuracy on the test set (tst2013.en and tst2013.vi). The training and validation loss curves over epochs for all configuration are reported in Appendix A.2.

Experiment Results. Table 5 summarizes the performance of all configurations in terms of accuracy, BLEU and perplexity. The results indicate that tokenization strategy has a dominant impact on translation quality, while embedding initialization and architectural changes provide incremental improvements.

Accuracy. The highest validation accuracy is achieved by Config C (0.623), followed closely by Configs A and D (0.620). The lowest accuracy is observed in the word-level configurations, with 0.550 in Config E and 0.555 in Config F.

Table 4: Experimental configurations

Config	Description
Baseline	Word-level tokenization using Keras tokenizer and ViTokenizer for Vietnamese preprocessing
A	SentencePiece tokenization with a shared English–Vietnamese subword vocabulary
B	SentencePiece tokenization with separate English and Vietnamese subword vocabularies
C	Config B with Skip-gram pretrained embeddings; embeddings frozen during the first two training epochs
D	Config C with SwiGLU replacing the ReLU-based feed-forward network
E	Whitespace tokenizer without subword segmentation
F	Whitespace tokenizer with SwiGLU-based feed-forward network

Compared to the baseline (0.583), SentencePiece-based models improve accuracy by up to +0.040, while word-level tokenization results in a drop of approximately -0.033. This gap suggests that subword modeling improves token-level prediction consistency, whereas word-level tokenization suffers from data sparsity and limited coverage of rare forms.

BLEU. The highest BLEU score is obtained by Config F (31.25), followed by Config E (30.10), both using word-level tokenization. In contrast, the lowest BLEU score is observed in Config A (17.96). Among subword-based models, BLEU increases monotonically from Config A to Config D (17.96 to 30.15), yielding a total gain of +11.99 BLEU. The large BLEU gap between word-level and subword-based models can be attributed to tokenization granularity, as word-level tokenization favors exact surface-form matches, which directly influence n-gram overlap used in BLEU computation.

Perplexity (PPL). PPL values exhibit an inverse trend relative to BLEU, with lower values indicating better predictive confidence. The lowest PPL is achieved by Config E (10.10) and Config F (10.12), while the highest PPL is observed in Config B (21.57) and Config D (21.24). Compared to the baseline (11.09), SentencePiece-based models roughly double PPL, reflecting increased uncertainty due to subword decomposition and longer effective sequences. Conversely, word-level models yield lower PPL because probabilities are assigned to whole words, resulting in sharper distributions despite weaker generalization.

3.6 Ablation Study

To better understand the contribution of each method, we conduct an ablation study by progressively modifying the baseline system.

Tokenization Strategy. Comparing Config A with the baseline shows that using SentencePiece with a shared vocabulary increases accuracy from 0.583 to 0.620, but reduces BLEU from 19.12 to 17.96. When separate vocabularies are used in Config B, BLEU increases substantially by 10.67 points, while accuracy remains nearly unchanged. This suggests that language-specific subword vocabularies are more suitable for English–Vietnamese translation.

Embedding Initialization. Moving from Config B to Config C, initializing embeddings with Skip-gram leads to consistent improvements in both BLEU (+1.23) and accuracy (+0.006), along with a small reduction in perplexity. These results indicate that pretrained embeddings provide useful lexical information that complements the parallel training data.

Architectural Modification (SwiGLU). In terms of feed-forward network, replacing ReLU with SwiGLU in Config D results in only a slight BLEU improvement (+0.09), while accuracy and perplexity remain largely unchanged. This suggests that the effect of SwiGLU is limited once tokenization and embedding initialization are already well optimized.

Word-level Ablation. In the word-level setting, adding SwiGLU (Config F) improves BLEU by 4.85 points compared to Config E, whereas accuracy increases only marginally (+0.005). This indicates that architectural changes can improve surface-level translation quality, but do not fully address the limitations of word-level tokenization.

Whitespace Better Performance. In the IWSLT'15 English–Vietnamese setting (about 133k sentence pairs, TED-talk domain, short utterances), whitespace tokenizer consistently outperforms SentencePiece. This observation is primarily explained by the interaction between Vietnamese linguistic characteristics, limited training data, model capacity, and BLEU

Table 5: Performance of different configurations on the ***tst2013*** test set. Higher accuracy and BLEU are better, while lower perplexity indicates more stable predictions.

Config	Method	Accuracy	BLEU	Perplexity
Baseline	Keras Tokenizer + ViTokenizer	0.583	19.120	11.09
A	SentencePiece (Shared vocab)	0.620	17.960	20.50
B	SentencePiece (Separate vocab)	0.617	28.633	21.57
C	SentencePiece + Skip-gram	**0.623**	29.858	20.97
D	SentencePiece + Skip-gram + SwiGLU	0.620	30.150	21.24
E	Whitespace Tokenizer	0.550	30.103	**10.10**
F	**Whitespace Tokenizer + SwiGLU**	0.555	**31.250**	10.12

evaluation. Vietnamese is written with whitespace-separated syllables that function similarly to meaningful subword units, making whitespace tokenization an effective implicit subword strategy. In contrast, SentencePiece may fragment coherent semantic units on the English side, as illustrated below:

Origin: environmental protection

Whitespace:

[environmental, protection]

SentencePiece:

_environ ment al _protect ion

In terms of SentencePiece, such tokenization increases sequence length and compositional complexity, which is detrimental for small models trained on limited data. Furthermore, BLEU evaluates word-level n-gram overlap after detokenization, tending to penalize subword-based outputs more heavily.

4 Part II: VLSP Medical Translation

In this section, we present our participation in the VLSP 2025 Shared Task ("Medical domain MT with Limited-Pretraining models"). We employed Instruction Tuning on a pre-trained Large Language Model (LLM) to leverage its vast linguistic knowledge and generalization capabilities.

4.1 Data Preprocessing and Knowledge Injection

To address the challenges of noise in given data and the high precision requirements of the medical domain, we implemented a rigorous data pipeline consisting of cleaning, sampling, and domain-specific knowledge injection.

4.1.1 Data Cleaning and Normalization

The initial dataset provided by VLSP contained approximately 500,000 sentence pairs. We applied a filtering pipeline to remove low-quality samples, including:

- Removing malformed sentences and encoding errors.
- Normalizing special mathematical and medical symbols (e.g., $\geq, \pm, \mu g$) to ensuring token consistency.
- Correcting spacing errors and unifying decimal formats (e.g., $4.\ 2\% \rightarrow 4.2\%$; $3,5\% \rightarrow 3.5\%$) to mitigate tokenization fragmentation and improve numerical reasoning.

This process resulted in a refined corpus of 498,558 high-quality parallel pairs.

4.1.2 Strategic Sampling

Given compute constraints and the need for high-quality instruction tuning, we performed a random

stratified sampling from the cleaned corpus to select a subset of 100,000 representative pairs. These pairs serve as the backbone of our training data.

4.1.3 Domain Knowledge Injection (Glossary and Abbreviations)

To prevent common translation errors related to specific terminology and clinical shorthand, we constructed two auxiliary knowledge bases:

- Medical Glossary (K_{gloss}): A curated lexicon of 500 critical English-Vietnamese medical terms.
- Abbreviation Map (K_{abbr}): A mapping of 65 common Vietnamese clinical abbreviations (e.g., "VXCC"→ "Viêm xương chũm cấp") to their full forms.

Using a Template-based Data Synthesis approach, we generated synthetic instruction samples from K_{gloss} and K_{abbr}. This forces the model to memorize exact terminology and resolve abbreviations explicitly during inference.

4.1.4 Final Instruction Dataset Construction

The final dataset was composed by combining the bidirectional translation tasks of the sampled corpus with the synthetic knowledge data:

$$D_{final} = D_{En \leftrightarrow Vi} \cup D_{Synthetic} \quad (1)$$

- Bidirectional Translation (100k pairs $\times$ 2): 200,000 samples.
- Synthetic Terminology Instructions: 51,460 samples.

Total Training Size: 251,460 samples.

4.2 Methodology: Fine-tuning Qwen

4.2.1 Base Model Selection

We selected the Qwen2.5-3B-Instruct as the backbone architecture for our specialized medical translation system. This model represents a strategic "sweet spot" in the landscape of Large Language Models (LLMs), balancing reasoning capability with computational feasibility. Our selection was driven by three key advantages:

Instruction Following Capability. The "Instruct" variant is aligned with human intent via Reinforcement Learning from Human Feedback (RLHF). This is critical for enforcing our strict two-stage pipeline (Draft → Refine) and ensuring the model adheres to specific formatting rules (Huang et al., 2025).

Vietnamese Proficiency and Domain Knowledge. Compared to smaller variants (e.g., 1.5B) or other open-source models (LLaMA-3-8B), Qwen-3B demonstrates significantly stronger zero-shot performance in Vietnamese and a deeper understanding of technical terminology. Its pre-training corpus covers a vast amount of multilingual data, essential for cross-lingual medical alignment (Zhu et al., 2025).

Optimized Efficiency. With 3 billion parameters, the model is substantial enough to capture complex grammatical nuances yet remains deployable on single-consumer GPUs (Qwen et al., 2025).

4.2.2 Parameter-Efficient Fine-tuning (PEFT)

To adapt the general-purpose Qwen-3B model to the high-precision requirements of the medical domain, we employed Low-Rank Adaptation (LoRA) (Hu et al., 2021). This technique allows us to inject domain knowledge without the catastrophic forgetting often associated with full fine-tuning.

Mathematical Formulation. Instead of updating the massive pre-trained weight matrix $W_0 \in R^{d \times k}$, LoRA freezes W_0 and injects trainable rank decomposition matrices. The forward pass is defined as:

$$h = W_0 x + \frac{\alpha}{r} BAx \quad (2)$$

Where:

- W_0 denotes the frozen Qwen-3B parameters (quantized to 4-bit via NF4 (Dettmers et al., 2023)).
- $B \in R^{d \times r}$ and $A \in R^{r \times k}$ are trainable adapters initialized with $r = 64$.
- α is a scaling factor (set to 128).

Training Objective. The model is optimized using the standard Causal Language Modeling (CLM) (Radford et al., 2019) objective on our augmented instruction dataset. Given the instruction I (including system prompts and glossary constraints) and source sentence S, we maximize the log-likelihood of the target translation T:

$$\mathcal{L}(\Phi) = -\sum_{t=1}^{|T|} \log P(y_t | y_{<t}, I, S; \Phi) \quad (3)$$

Table 6: Examples of Training Data Samples after formatting (Instruction-based).

Type	Instruction	Input	Output
Translation (Vi $\rightarrow$ En)	You are a professional medical translator. Translate the following Vietnamese text to English.	Bệnh nhân nhập viện vì đau ngực trái dữ dội.	The patient was admitted due to severe left chest pain.
Synthetic (Abbreviation)	Explain the Vietnamese medical abbreviation in the input context.	Bệnh nhân được chẩn đoán bị **VXCC**.	In this context, "VXCC" stands for "Viêm xương chũm cấp" (Acute mastoiditis).

This objective ensures the model not only translates accurately but also strictly complies with the explicit constraints defined in our prompts.

4.2.3 Implementation Details & Hyperparameters

We implemented the fine-tuning pipeline using the Unsloth library, a high-performance framework that optimizes backpropagation kernels to increase training speed by up to 2x and reduce memory usage by 60% compared to standard HuggingFace implementations. The training was conducted on a single NVIDIA RTX A6000 GPU.

QLoRA Configuration: To enable fine-tuning of the 3B model within limited VRAM while maximizing learning capacity, we employed 4-bit Normal Float (NF4) quantization. Crucially, we configured the Low-Rank Adaptation (LoRA) with a high rank ($r = 64$) to capture complex cross-lingual medical mappings:

- Rank (r): 64 (Higher capacity than the standard $r = 16$).
- Scaling Factor (α): 128 (Following the convention $\alpha = 2r$ to stabilize updates).
- Target Modules: We applied LoRA adapters to *all* linear layers in the Transformer blocks: query (q_proj), key (k_proj), value (v_proj), output (o_proj), and the MLP projections ($gate, up, down\_proj$). This ensures comprehensive adaptation of the model's attention and reasoning mechanisms.
- Dropout: 0.0 (To preserve deterministic behavior during training).

Training Hyperparameters. Optimization was performed using the AdamW 8-bit optimizer to further reduce memory footprint regarding optimizer states. The detailed configuration is presented in Table 7.

Table 7: Hyperparameter Settings derived from empirical tuning.

Hyperparameter	Value
Learning Rate	2×10^{-4}
Scheduler	Linear Decay
Warmup Steps	100
Per-device Batch Size	32
Gradient Accumulation	2
Effective Batch Size	64
Max Sequence Length	2048 tokens
Num Epochs	1
Optimizer	AdamW (8-bit)

4.3 Experimental Setup (Part II)

To investigate the impact of model scale and resource constraints on translation performance, we adopted an Iterative Experimental Strategy. Our evaluation process was conducted in two distinct phases, progressing from a resource-constrained environment to a high-performance compute node.

4.3.1 Phase 1: Resource-Constrained Evaluation

Initial experiments were conducted on the Kaggle cloud platform using the Qwen2.5-1.5B model.

- Infrastructure: Single NVIDIA Tesla T4 (16GB VRAM).

- Configuration: Due to memory constraints, we used a conservative LoRA configuration (Rank $r = 16$, Alpha $\alpha = 32$) and a shorter sequence length (1024 tokens).
- Purpose: This phase served as an ablation study to optimize hyperparameters and validate the data pipeline. While efficient, the 1.5B model showed limitations in handling complex medical syntax.

4.3.2 Phase 2: Full-Scale Optimization

For the final system, we scaled up to the Qwen2.5-3B model, deployed on a high-performance local workstation to maximize translation quality.

- Infrastructure: Single NVIDIA RTX A6000 (48GB VRAM).
- Advanced Configuration: Leveraging the massive VRAM, we aggressively increased the model capacity:
  - Model: Qwen2.5-3B-Instruct (4-bit quantization).
  - LoRA Rank (r): Increased to 64 (4x larger than Phase 1) to capture deep semantic nuances.
  - Target Modules: All linear layers ($q, k, v, o, gate, up, down\_proj$).
  - Sequence Length: Extended to 2048 tokens to accommodate long medical records.

Final Hyperparameter Configuration. The comparison in settings between two phases are detailed in Table 8:

Evaluation Metrics. Strictly adhering to the VLSP 2025 Shared Task guidelines, we employed a hybrid evaluation protocol that synergizes traditional n-gram metrics with advanced LLM-based assessment. For lexical precision, the BLEU score was computed via sacrebleu on the official public test set, with case-sensitive evaluation enforced to guarantee exact matches for capitalized medical terminology. Complementing this, we utilized Gemini 1.5 Flash as a judge to measure semantic fidelity, with a score on a scale of 1 to 100 based on three critical dimensions: *Accuracy* (preservation of clinical meaning and dosage), *Fluency* (grammatical correctness), and *Terminology* (adherence to the specialized medical glossary).

4.4 Results and Analysis (Part II)

To investigate the impact of model scaling and inference strategies on translation quality, we conducted a comparative evaluation on both Qwen-1.5B and Qwen-3B architectures. We assessed performance across different configurations, ranging from zero-shot baselines to our advanced refinement pipelines.

4.4.1 Quantitative Results

Table 9 presents the performance metrics on BLEU and Gemini Score for both model sizes.

Analysis of Quantitative Results. The results presented in Table 9 demonstrate the progression of performance across two phases of experimentation, highlighting the impact of model scaling and specialized refinement strategies.

Phase 1: Impact of Constraints on Smaller Models. The baseline 1.5B model exhibited severe generative instability, characterized by extreme verbosity (Ratio 3.36) and low translation quality (BLEU 7.30). However, the implementation of *Fine-tuned + Glossary Constraint* effectively curbed this hallucination, stabilizing the length ratio to 1.10 and significantly boosting the BLEU score to 25.25.

Phase 2: Scaling Efficiency. Scaling to the 3B parameter space provided immediate gains. The *Fine-tuned (Standard)* configuration achieved a near-perfect length ratio (1.01) and demonstrated a substantial leap in performance compared to the 1.5B counterpart, raising the BLEU score (En→Vi) to 41.18.

Superiority of the Refine Pipeline. The proposed *Fine-tuned + Refine Pipeline* (Row 5) yielded the state-of-the-art results across all metrics. It achieved the highest BLEU score of **44.00** and maximized semantic accuracy as reflected by the Gemini Score (reaching $\approx$ **87.90** for En→Vi). This confirms that adding a refinement stage post-finetuning is critical for handling complex medical terminology while maintaining fluency.

4.4.2 Qualitative Analysis & Case Studies

To complement the quantitative metrics, we conducted a manual inspection of the generated outputs to understand the specific improvements introduced by our Refine Pipeline.

Mitigation of Generative Artifacts. The high length ratio (3.36) observed in the Phase 1 Baseline was primarily caused by repetition loops, where the

Table 8: Experimental Configuration Comparison Between Resource-Constrained Setup and Full-Scale Optimization.

Parameter	**Phase 1: Resource-Constrained**	**Phase 2: Full-Scale**
Model Architecture	Qwen2.5-1.5B-Instruct	Qwen2.5-3B-Instruct
Compute Infrastructure	Single Tesla T4 (16GB)	Single RTX A6000 (48GB)
Dataset Size	$\sim$500k samples (raw, noisy)	$\sim$251k samples (curated & augmented)
Training Schedule	Fixed steps (2,000 steps)	Full epoch ($\sim$3,900 steps)
LoRA Rank (r)	16	64
LoRA Scaling (α)	32	128
Target Modules	Attention layers only	All linear layers
Maximum Sequence Length	1,024 tokens	2,048 tokens
Optimizer	AdamW (8-bit)	AdamW (8-bit)
Effective Batch Size	16	64

Table 9: Comprehensive Performance Comparison: Qwen-1.5B vs. Qwen-3B across different strategies.

Model Architecture	**Strategy / Configuration**	**BLEU (En→Vi)**	**BLEU (Vi→En)**	**Ratio**	**Gemini (En→Vi)**	**Gemini (Vi→En)**
I. Phase 1 (Qwen2.5-1.5B-Instruct)						
Qwen-1.5B	*Baseline (Zero-shot)*	7.30	10.97	3.36	14.58	21.80
	Fine-tuned + Glossary Constraint	25.25	19.37	1.10	50.44	38.68
II. Phase 2 (Qwen2.5-3B-Instruct)						
Qwen-3B	*Baseline (Zero-shot)*	21.71	18.34	1.43	43.37	36.64
	Fine-tuned (Standard)	41.18	30.10	1.01	85.04	81.93
	Fine-tuned + Refine Pipeline	**44.00**	**35.46**	**1.01**	**87.90**	**84.47**

Note: "Ratio" indicates the length ratio between hypothesis and reference. Gemini Score are evaluated on a scale of 1-100.

model failed to terminate generation and repeated the last phrase indefinitely. As illustrated in Table 9 (Phase 1), the Zero-shot model often hallucinated redundant explanations. Our optimization strategy effectively suppressed these artifacts, aligning the output length with the reference (Ratio $\approx$ 1.01) without truncating essential information.

Terminological Consistency via Refinement. While standard fine-tuning improved general fluency, it occasionally struggled with ambiguous medical entities. Comparing (Standard) and (Refine Pipeline), we observed that the inclusion of the refinement stage acted as a semantic safeguard. For instance, in complex diagnostic sentences, the Refine Pipeline correctly mapped specific acronyms and drug names according to the glossary, whereas the standard model tended to output generic or literal translations. This qualitative leap justifies the higher Gemini Score observed in the final configuration.

Robustness on Short-Form Inputs. A critical failure mode observed in the earlier models was the omission of short sentences (under-translation). The Baseline often treated brief, standalone segments as noise and skipped them entirely. The Refine Pipeline demonstrated superior coverage, ensuring that even minimal segments were translated with the same fidelity as longer contexts, effectively eliminating this data loss.

4.4.3 Discussion

Our findings indicate that while model scaling (from 1.5B to 3B) naturally improves reasoning capabilities, it is the combination of Glossary Constraints and the Refine Pipeline that ensures terminological precision. The transition from a Ratio of 3.36 to 1.01, coupled with the highest performance across both BLEU and Gemini metrics, validates our hypothesis that controlled generation is essential for high-stakes domains like medical translation.

5 Conclusion

Task 1. Our study on the IWSLT'15 dataset demonstrates that tokenization strategy is the decisive factor for English-Vietnamese NMT, significantly outweighing other architectural modifications. We found that simple whitespace tokenization consistently

outperforms SentencePiece, achieving the best result (BLEU 31.25) when combined with SwiGLU. This confirms that for low-resource scenarios involving Vietnamese, preserving natural syllable boundaries is superior to aggressive subword segmentation, while techniques like Skip-gram embeddings and SwiGLU serve as effective but supplementary enhancements. **Task 2.** We presented a resource-efficient framework for the VLSP 2025 Medical Translation task, demonstrating that model size is secondary to controlled generation strategies. While scaling to 3B parameters offered natural improvements, our proposed Refine Pipeline proved to be the decisive factor, boosting BLEU to **44.00** and effectively resolving generative hallucinations (stabilizing length ratio to **1.01**). This confirms that for specialized high-stakes domains, combining parameter-efficient fine-tuning with inference-time glossary constraints is the optimal strategy to ensure terminological precision without relying on massive computational resources.

6 Future Work

Looking forward, we identify several promising directions to further enhance system performance. First, we plan to integrate Retrieval-Augmented Generation (RAG) to dynamically retrieve terminology from external medical dictionaries during inference, thereby mitigating errors related to rare drug names. Second, we aim to explore Model Ensembling techniques, combining outputs from multiple LoRA checkpoints to improve translation stability. Finally, investigating Iterative Back-translation could further enrich the training corpus, helping the model generalize better on complex sentence structures.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450*.

Mauro Cettolo, Jan Niehues, Sebastian Stüker, Luisa Bentivogli, Roldano Cattoni, and Marcello Federico. 2015. The IWSLT 2015 evaluation campaign. In *Proceedings of the 12th International Workshop on Spoken Language Translation: Evaluation Campaign*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Preprint*, arXiv:2305.14314.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Shulin Huang, Yiran Ding, Junshu Pan, and Yue Zhang. 2025. Beyond english-centric training: How reinforcement learning improves cross-lingual reasoning in llms. *Preprint*, arXiv:2509.23657.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners.

Noam Shazeer. 2020. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need.

Hengchuan Zhu, Yihuan Xu, Yichen Li, Zijie Meng, and Zuozhu Liu. 2025. Dentalbench: Benchmarking and advancing llms capability for bilingual dentistry understanding. *Preprint*, arXiv:2508.20416.

A Appendix

A.1 Noam Learning Rate Scheduler.

The Noam learning rate scheduler, introduced in Attention Is All You Need (Vaswani et al., 2023), is designed for training Transformers from scratch. It increases the learning rate linearly during a warmup phase and then decays it with the inverse square root of the training step to ensure stable convergence. The learning rate at step t is defined as:

$$lr(t) = d_{model}^{-0.5} \cdot \min\left(t^{-0.5},\ t \cdot warmup^{-1.5}\right),$$

where d_{model} denotes the model dimensionality, t is the current training step, and $warmup$ is the

number of warmup steps. This scheduling strategy has been shown to significantly improve training stability and convergence for deep Transformer architectures.

A.2 Training Loss Curves Across Experimental Configurations

This section presents the training loss curves for the four experimental configurations to analyze their convergence behavior and optimization stability. Comparing these curves helps illustrate the impact of different tokenization strategies, embedding initialization, and architectural modifications on training dynamics.

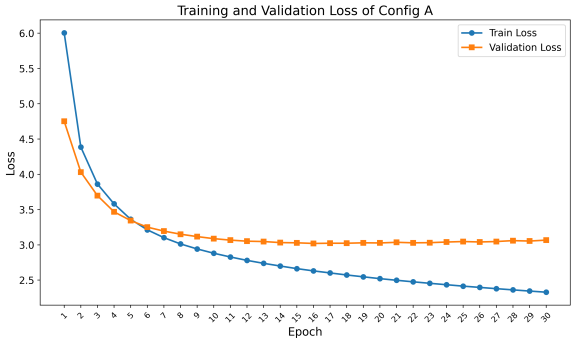

Figure 8: Training loss curve for $Config_A$.

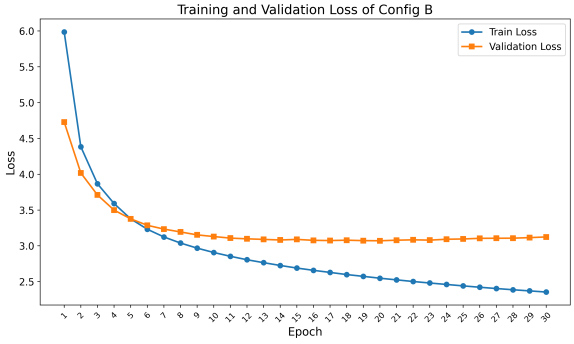

Figure 9: Training loss curve for $Config_B$.

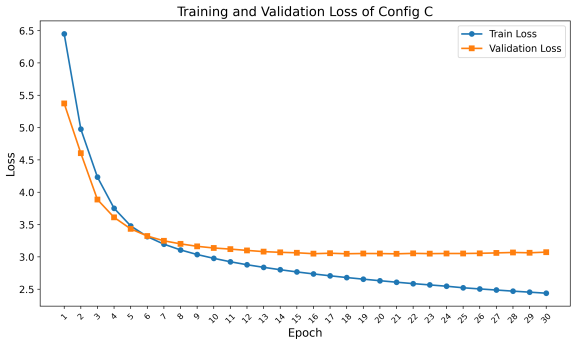

Figure 10: Training loss curve for $Config_C$.

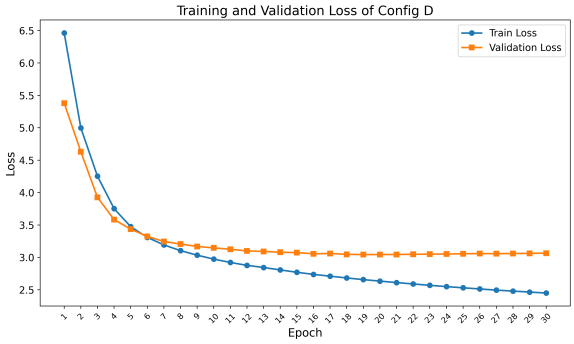

Figure 11: Training loss curve for $Config_D$.

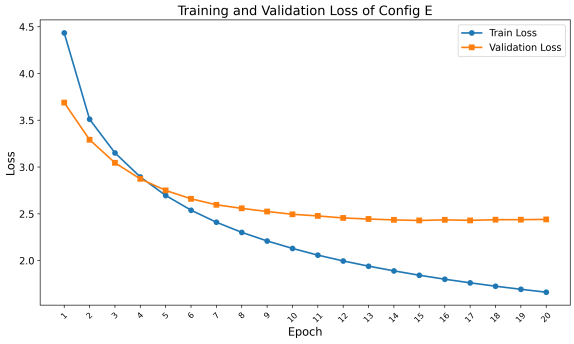

Figure 12: Training loss curve for $Config_E$.

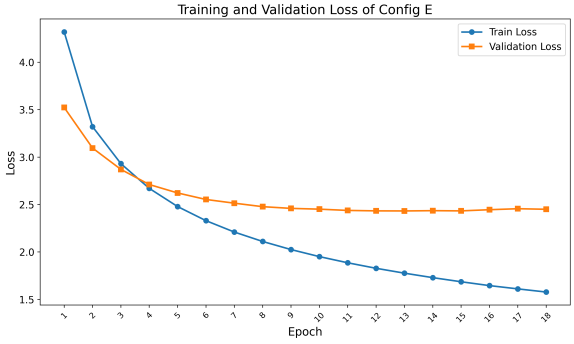

Figure 13: Training loss curve for $Config_F$.